Selected Programming Topics

Prepared by: Yashaswini

Project ideas with Python starter code

This document lists ten selected topics and includes a simple Python starter program for each topic. The programs demonstrate the core logic and can be extended for a college assignment.

1. University Scholarship Allocation

Uses academic percentage and annual family income to classify a student under sample scholarship rules.

```
def scholarship(percentage, annual_income):
    if percentage >= 90 and annual_income <= 200000:
        return "Full scholarship"
    elif percentage >= 80 and annual_income <= 400000:
        return "Partial scholarship"
    return "Not eligible"

marks = float(input("Enter percentage: "))
income = float(input("Enter annual family income (■): "))
print(scholarship(marks, income))
```

2. Progressive Electricity Billing

Calculates a bill using different rates for different unit ranges. Rates are examples only.

```
def electricity_bill(units):
    if units <= 100:
        return units * 2.0
    elif units <= 200:
        return 100 * 2.0 + (units - 100) * 3.5
    else:
        return 100 * 2.0 + 100 * 3.5 + (units - 200) * 5.0

units = float(input("Enter electricity units used: "))
if units < 0:
    print("Units cannot be negative.")
else:
    print(f"Electricity bill: ■{electricity_bill(units):.2f}")
```

3. Employee Payroll and Deduction Analysis

Shows gross pay, example deductions, and net pay. Replace sample rates with the rules provided by your assignment.

```
def calculate_pay(basic_salary):
    allowance = basic_salary * 0.20
    tax = basic_salary * 0.05
    provident_fund = basic_salary * 0.12
    gross = basic_salary + allowance
    deductions = tax + provident_fund
    return gross, deductions, gross - deductions

salary = float(input("Enter basic monthly salary (■): "))
if salary < 0:
    print("Salary cannot be negative.")
else:
    gross, deductions, net = calculate_pay(salary)
    print(f"Gross pay: ■{gross:.2f}")
    print(f"Total deductions: ■{deductions:.2f}")
    print(f"Net pay: ■{net:.2f}")
```

4. Retail Billing with Multiple Discount Rules

Applies a purchase-value discount and a membership discount, capped at 20%.

```
def final_price(price, quantity, member):
    total = price * quantity
    discount = 0.10 if total >= 5000 else 0.05 if total >= 2000 else 0
    if member:
        discount += 0.02
    discount = min(discount, 0.20)
    return total * (1 - discount)

price = float(input("Enter item price (■): "))
quantity = int(input("Enter quantity: "))
member = input("Member customer? (yes/no): ").strip().lower() == "yes"

if price < 0 or quantity < 0:
    print("Price and quantity cannot be negative.")
else:
    print(f"Final bill: ■{final_price(price, quantity, member):.2f}")
```

5. Attendance Recovery Computation

Estimates how many consecutive classes are needed to reach a target attendance percentage.

```
def required_attendance(current_classes, attended, target=75):
    if current_classes <= 0 or attended < 0 or attended > current_classes:
        return None
    needed = target / 100 * current_classes - attended
    if needed <= 0:
        return 0
    # Solve (attended + x) / (current_classes + x) >= target / 100
    return int(needed / (1 - target / 100)) + (1 if needed % (1 - target / 100) else 0)

total = int(input("Classes conducted so far: "))
attended = int(input("Classes attended: "))
result = required_attendance(total, attended)
if result is None:
    print("Please enter valid attendance values.")
else:
    print("Additional consecutive classes to attend:", result)
```

6. Warehouse Inventory Reconciliation

Compares expected stock with recorded closing stock to find shortages or surpluses.

```
def reconcile(opening_stock, received, dispatched, recorded_closing):
    expected = opening_stock + received - dispatched
    difference = recorded_closing - expected
    return expected, difference

opening = int(input("Opening stock: "))
received = int(input("Stock received: "))
dispatched = int(input("Stock dispatched: "))
closing = int(input("Recorded closing stock: "))

if min(opening, received, dispatched, closing) < 0:
    print("Stock values cannot be negative.")
else:
    expected, difference = reconcile(opening, received, dispatched, closing)
    print("Expected closing stock:", expected)
    print("Difference (recorded - expected):", difference)
    print("Status:", "Matched" if difference == 0 else "Needs checking")
```

7. Student Marks Matrix Analysis

Stores marks in a two-dimensional list and calculates each student's total and average.

```
def analyze_marks(matrix):
    for i, row in enumerate(matrix, start=1):
        average = sum(row) / len(row)
        print(f"Student {i}: total={sum(row)}, average={average:.2f}")

students = int(input("Number of students: "))
subjects = int(input("Number of subjects: "))
if students <= 0 or subjects <= 0:
    print("Counts must be positive.")
else:
    marks = []
    for i in range(students):
        row = []
        for j in range(subjects):
            mark = float(input(f"Student {i+1}, subject {j+1} mark: "))
            row.append(mark)
        marks.append(row)
    analyze_marks(marks)
```

8. Sports Tournament Ranking System

Ranks teams by points, then by score difference as a tie-breaker.

```
def rank_teams(results):
    # results: list of (team_name, points, score_difference)
```

```
    return sorted(results, key=lambda item: (item[1], item[2]), reverse=True)

teams = []
count = int(input("Number of teams: "))
for _ in range(count):
    name = input("Team name: ")
    points = int(input("Points: "))
    difference = int(input("Score difference: "))
    teams.append((name, points, difference))

for position, team in enumerate(rank_teams(teams), start=1):
    print(position, team[0], "- points:", team[1], "- score difference:", team[2])
```

9. Smart Canteen Order and Stock Management

Accepts orders, checks available stock, updates inventory, and prints the total.

```
menu = {
    "idli": {"price": 30, "stock": 20},
    "dosa": {"price": 50, "stock": 15},
    "juice": {"price": 40, "stock": 10}
}

total = 0
while True:
    item = input("Enter item (idli/dosa/juice) or 'done': ").strip().lower()
    if item == "done":
        break
    if item not in menu:
        print("Item not on menu.")
        continue
    quantity = int(input("Quantity: "))
    if quantity <= 0:
        print("Quantity must be positive.")
    elif quantity > menu[item]["stock"]:
        print("Not enough stock.")
    else:
        menu[item]["stock"] -= quantity
        total += menu[item]["price"] * quantity
        print("Added to order.")
print(f"Total bill: ■{total:.2f}")
print("Remaining stock:", {item: data["stock"] for item, data in menu.items()})
```

10. Student Academic Risk Classification

Uses example thresholds to flag students who may need academic support. It should support, not replace, teacher judgement.

```
def classify_risk(attendance, average_mark, missing_assignments):
    if attendance < 60 or average_mark < 40 or missing_assignments >= 4:
        return "High risk"
    elif attendance < 75 or average_mark < 55 or missing_assignments >= 2:
        return "Moderate risk"
    return "Low risk"

attendance = float(input("Attendance percentage: "))
average = float(input("Average marks percentage: "))
missing = int(input("Number of missing assignments: "))

if not (0 <= attendance <= 100 and 0 <= average <= 100 and missing >= 0):
    print("Enter valid values.")
else:
    print("Academic risk:", classify_risk(attendance, average, missing))
```

Note: These are educational examples. Confirm the exact rules, thresholds, and rates required by your teacher before submitting.